

1) To create three promises where the first two are rejected and the last one is resolved, you can chain them or manage them independently

To create three promises where the first two are rejected and the last one is resolved, you can chain them or manage them independently. Here's an example using three separate promises:

```
javascript Copy code

const promise1 = new Promise((resolve, reject) => {
  setTimeout(() => reject('Promise 1 rejected'), 1000);
});

const promise2 = new Promise((resolve, reject) => {
  setTimeout(() => reject('Promise 2 rejected'), 2000);
});

const promise3 = new Promise((resolve, reject) => {
  setTimeout(() => resolve('Promise 3 resolved'), 3000);
});

// To handle the promises sequentially
promise1
  .catch(error => {
    console.log(error);
    return promise2;
  })
  .catch(error => {
    console.log(error);
    return promise3;
  })
  .then(result => {
    console.log(result);
  })
  .catch(error => {
    console.log('This will not be called because the last promise resolved');
  });
```

Explanation:

- **Promise 1:** Rejects after 1 second with the message "Promise 1 rejected".
- **Promise 2:** Rejects after 2 seconds with the message "Promise 2 rejected".

- **Promise 3:** Resolves after 3 seconds with the message "Promise 3 resolved".

Execution Flow:

1. **Promise 1:** The first promise is rejected, and its error is caught.
2. **Promise 2:** The second promise is also rejected, and its error is caught.
3. **Promise 3:** The third promise is resolved, and its result is logged to the console.

Output:

javascript

 Copy code

```
Promise 1 rejected
Promise 2 rejected
Promise 3 resolved
```

This setup allows the promises to execute in sequence, where each promise is handled only after the previous one has been rejected or resolved.

2) Most Frequent String

```
const arr1 = 'ababacdbe';
```

```
const result = mostFrequentElements(arr1);
```

```
console.log(result.mostFrequent); // Output: ['a', 'b']
```

```
console.log(result.maxCount); // Output: 3
```

```
const arr1 = 'ababacdbe';
const mostFrequentElements = (arr1) => {
  let arr = arr1.split('');
  const frequencyMap = new Map();
  let maxCount = 0;
  for (const elem of arr) {
    const count = (frequencyMap.get(elem) || 0) + 1;
    frequencyMap.set(elem, count);
    maxCount = Math.max(maxCount, count);
  }
  const mostFrequent = [...frequencyMap.entries()].filter(([elem, count]) => count === maxCount).map(([elem, count]) => elem);

  return {
    mostFrequent,
    maxCount
  };
};
const result = mostFrequentElements(arr1);
```

```
console.log(result.mostFrequent); // Output: ['a', 'b']
console.log(result.maxCount); // Output: 3
```

3) Concurrency in JavaScript

Yes, by using asynchronous programming, you can achieve concurrency in JavaScript. JavaScript is single-threaded, meaning it can only execute one task at a time per thread, but it can handle multiple tasks concurrently by using non-blocking, asynchronous operations.

Key Concepts for Achieving Concurrency in JavaScript:

1. Asynchronous Functions (`async/await`):

- `async` functions return a promise and allow you to write asynchronous code in a synchronous-looking manner using `await`.
- `await` pauses the execution of the function until the promise is resolved, without blocking the main thread.

javascript

 Copy code

```
async function fetchData() {
  const response = await fetch('https://api.example.com/data');
  const data = await response.json();
  console.log(data);
}
```

2. Promises:

- Promises are a core feature for handling asynchronous operations. They represent the eventual completion (or failure) of an asynchronous operation and its resulting value.
- You can chain `.then()` and `.catch()` methods to handle the result of a promise.

javascript

 Copy code

```
fetch('https://api.example.com/data')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error('Error:', error));
```

3. Callbacks:

- Before promises and `async/await` were introduced, callbacks were the primary way to handle asynchronous code in JavaScript.
- A function is passed as an argument to another function and is executed after the asynchronous operation completes.

javascript

 Copy code

```
setTimeout(() => {  
  console.log('This runs after 2 seconds');  
}, 2000);
```

4. Event Loop:

- The event loop is what allows JavaScript to perform non-blocking I/O operations. It continuously checks the call stack and the task queue, executing tasks when the stack is empty.

5. Web APIs:

- Functions like `setTimeout`, `setInterval`, `fetch`, and DOM event listeners are provided by the browser (or Node.js environment) and operate asynchronously, allowing other code to run while waiting for these operations to complete.

Example of Concurrency:

javascript

 Copy code

```
async function concurrentTasks() {  
  const task1 = fetch('https://api.example.com/data1');  
  const task2 = fetch('https://api.example.com/data2');  
  
  const [result1, result2] = await Promise.all([task1, task2]);  
  
  console.log('Result 1:', await result1.json());  
  console.log('Result 2:', await result2.json());  
}  
  
concurrentTasks();
```

In this example, both `task1` and `task2` are started concurrently, and the results are awaited simultaneously using `Promise.all()`. This allows for concurrent execution of asynchronous tasks, even though JavaScript is single-threaded.

4) Hoisting, Closures, and Promises

Hoisting, closures, and promises are important concepts in JavaScript that play key roles in understanding how code execution works, managing function scope, and handling asynchronous operations. Here's an overview of each concept:

1. Hoisting

Hoisting is a JavaScript mechanism where variable and function declarations are moved ("hoisted") to the top of their containing scope during the compile phase, before the code is executed.

Key Points:

- **Variable Hoisting:** Variables declared with `var` are hoisted to the top of their scope and initialized with `undefined`. Variables declared with `let` and `const` are also hoisted but are not initialized, leading to a `ReferenceError` if accessed before declaration (temporal dead zone).
- **Function Hoisting:** Function declarations are hoisted completely, meaning you can call a function before its definition in the code.

Example:

```
javascript Copy code

console.log(hoistedVar); // undefined
var hoistedVar = 'I am hoisted';

hoistedFunction(); // "I am a hoisted function"
function hoistedFunction() {
    console.log('I am a hoisted function');
}

// For let and const:
console.log(hoistedLet); // ReferenceError: Cannot access 'hoistedLet' be
let hoistedLet = 'I am not fully hoisted';
```

2. Closures

Closures are functions that have access to the variables from another function's scope, even after that function has finished executing. A closure is created when an inner function accesses variables from its outer function, allowing the inner

function to "remember" the outer function's variables even after the outer function has returned.

Key Points:

- Closures capture and remember the environment in which they were created.
- They are often used to create private variables or functions, as well as for currying and partial application.

Example:

```
javascript Copy code

function outerFunction(outerVariable) {
  return function innerFunction(innerVariable) {
    console.log('Outer Variable: ' + outerVariable);
    console.log('Inner Variable: ' + innerVariable);
  };
}

const newFunction = outerFunction('outside');
newFunction('inside');

// Output:
// Outer Variable: outside
// Inner Variable: inside
```

3. Promises

Promises are objects representing the eventual completion (or failure) of an asynchronous operation and its resulting value. Promises allow you to work with asynchronous code in a more manageable way, avoiding callback hell and providing methods like `.then()` and `.catch()` to handle success and errors, respectively.

Key Points:

- **States:** A promise can be in one of three states: `pending`, `fulfilled`, or `rejected`.
- **Chaining:** Promises can be chained with `.then()` to handle the result or `.catch()` to handle errors.
- **Async/Await:** Promises work well with `async` and `await` syntax for writing asynchronous code that looks synchronous.

Example:

javascript

 Copy code

```
// Creating a promise
const myPromise = new Promise((resolve, reject) => {
  setTimeout(() => {
    const success = true; // Simulating success or failure
    if (success) {
      resolve('Operation was successful');
    } else {
      reject('Operation failed');
    }
  }, 1000);
});

// Consuming a promise
myPromise
  .then((message) => {
    console.log(message); // "Operation was successful"
  })
  .catch((error) => {
    console.error(error);
  });

// Using async/await
async function runAsyncTask() {
  try {
    const result = await myPromise;
    console.log(result); // "Operation was successful"
  } catch (error) {
    console.error(error);
  }
}

runAsyncTask();
```

Summary:

- **Hoisting** allows function and variable declarations to be used before they appear in the code.
- **Closures** let functions access variables from an outer scope even after the outer function has returned.
- **Promises** provide a cleaner way to handle asynchronous operations, helping manage code that would otherwise rely on callbacks.

5) Arrays in JavaScript

JavaScript provides a rich set of array methods that allow you to manipulate and interact with arrays in various ways. Here's a comprehensive list of some commonly used array methods:

1. `push()`

- **Purpose:** Adds one or more elements to the end of an array and returns the new length of the array.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.push(4); // arr becomes [1, 2, 3, 4]
```

2. `pop()`

- **Purpose:** Removes the last element from an array and returns that element.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];  
let last = arr.pop(); // last is 3, arr becomes [1, 2]
```

3. `shift()`

- **Purpose:** Removes the first element from an array and returns that element.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];  
let first = arr.shift(); // first is 1, arr becomes [2, 3]
```

4. `unshift()`

- **Purpose:** Adds one or more elements to the beginning of an array and returns the new length of the array.
- **Example:**

javascript

 Copy code

```
let arr = [2, 3];  
arr.unshift(1); // arr becomes [1, 2, 3]
```

5. `concat()`

- **Purpose:** Merges two or more arrays into a new array.
- **Example:**

javascript

 Copy code

```
let arr1 = [1, 2];  
let arr2 = [3, 4];  
let combined = arr1.concat(arr2); // combined is [1, 2, 3, 4]
```

6. `slice()`

- **Purpose:** Returns a shallow copy of a portion of an array into a new array, selected from `start` to `end` (end not included).
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 4, 5];  
let sliced = arr.slice(1, 3); // sliced is [2, 3]
```

7. `splice()`

- **Purpose:** Changes the contents of an array by removing or replacing existing elements and/or adding new elements in place.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 4, 5];  
arr.splice(2, 1); // Removes 1 element at index 2, arr becomes [1, 2, 4, 5]  
arr.splice(2, 0, 3); // Inserts 3 at index 2, arr becomes [1, 2, 3, 4, 5]
```

8. `forEach()`

- **Purpose:** Executes a provided function once for each array element.

- Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.forEach(element => console.log(element)); // Logs 1, 2, 3
```

9. `map()`

- **Purpose:** Creates a new array populated with the results of calling a provided function on every element in the calling array.
- Example:

javascript

 Copy code

```
let arr = [1, 2, 3];  
let doubled = arr.map(x => x * 2); // doubled is [2, 4, 6]
```

10. `filter()`

- **Purpose:** Creates a new array with all elements that pass the test implemented by the provided function.
- Example:

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let evens = arr.filter(x => x % 2 === 0); // evens is [2, 4]
```

11. `reduce()`

- **Purpose:** Executes a reducer function on each element of the array, resulting in a single output value.
- Example:

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let sum = arr.reduce((acc, cur) => acc + cur, 0); // sum is 10
```

12. `find()`

- **Purpose:** Returns the value of the first element in the array that satisfies the provided testing function. Otherwise, it returns `undefined`.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let found = arr.find(x => x > 2); // found is 3
```

13. `findIndex()`

- **Purpose:** Returns the index of the first element in the array that satisfies the provided testing function. Otherwise, it returns `-1`.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let index = arr.findIndex(x => x > 2); // index is 2
```

14. `some()`

- **Purpose:** Tests whether at least one element in the array passes the test implemented by the provided function. It returns a Boolean value.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
let hasEven = arr.some(x => x % 2 === 0); // hasEven is true
```

15. `every()`

- **Purpose:** Tests whether all elements in the array pass the test implemented by the provided function. It returns a Boolean value.
- **Example:**

javascript

 Copy code

```
let arr = [2, 4, 6];  
let allEven = arr.every(x => x % 2 === 0); // allEven is true
```

16. `sort()`

- **Purpose:** Sorts the elements of an array in place and returns the sorted array.
- **Example:**

javascript

 Copy code

```
let arr = [4, 2, 3, 1];  
arr.sort(); // arr becomes [1, 2, 3, 4]
```

17. `reverse()`

- **Purpose:** Reverses the order of the elements in an array in place.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];  
arr.reverse(); // arr becomes [3, 2, 1]
```

18. `includes()`

- **Purpose:** Determines whether an array includes a certain value among its entries, returning `true` or `false` as appropriate.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];  
let containsTwo = arr.includes(2); // containsTwo is true
```

19. `join()`

- **Purpose:** Joins all elements of an array into a string.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];  
let str = arr.join('-'); // str is "1-2-3"
```

20. `flat()`

- **Purpose:** Creates a new array with all sub-array elements concatenated into it recursively up to the specified depth.
- **Example:**

javascript

 Copy code

```
let arr = [1, [2, 3], [4, [5, 6]]];  
let flatArray = arr.flat(2); // flatArray is [1, 2, 3, 4, 5, 6]
```

21. `flatMap()`

- **Purpose:** Maps each element using a mapping function, then flattens the result into a new array.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];  
let mappedFlatArray = arr.flatMap(x => [x * 2]); // mappedFlatArray is [2, 4, 6]
```

22. `fill()`

- **Purpose:** Fills all the elements of an array from a start index to an end index with a static value.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 4];  
arr.fill(0, 1, 3); // arr becomes [1, 0, 0, 4]
```

23. `copyWithin()`

- **Purpose:** Shallow copies part of an array to another location in the same array and returns it, without modifying its length.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 4, 5];  
arr.copyWithin(0, 3); // arr becomes [4, 5, 3, 4, 5]
```

24. `indexOf()`

- **Purpose:** Returns the first index at which a given element can be found in the array, or `-1` if it is not present.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3];
let index = arr.indexOf(2); // index is 1
```

25. `lastIndexOf()`

- **Purpose:** Returns the last index at which a given element can be found in the array, or `-1` if it is not present.
- **Example:**

javascript

 Copy code

```
let arr = [1, 2, 3, 2];
let lastIndex = arr.lastIndexOf(2); // lastIndex is 3
```

These methods provide powerful ways to work with arrays in JavaScript, offering a variety of ways to manipulate, traverse, and transform array data.

6) Javascript Objects

In JavaScript, objects are a collection of key-value pairs, where the keys are known as properties, and the values can be of any data type (including functions, known as methods when associated with an object).

Creating an Object

1. Object Literals

This is the most common way to create an object.

 Copy code

```
const person = {
  name: "John",
  age: 30,
  greet: function() {
    console.log("Hello, " + this.name);
  }
}
```

```
}  
};  
  
console.log(person.name); // Output: John  
person.greet(); // Output: Hello, John
```

In this example:

- `name` and `age` are properties (keys), and `"John"` and `30` are their values.
- `greet` is a method (a function defined within the object).

2. Using `new Object()`

This is another way to create an object, but it is less commonly used.

 Copy code

```
const person = new Object();  
person.name = "John";  
person.age = 30;  
person.greet = function() {  
  console.log("Hello, " + this.name);  
};  
  
console.log(person.name); // Output: John  
person.greet(); // Output: Hello, John
```

Accessing Object Properties

1. Dot Notation:

 Copy code

```
console.log(person.name); // Output: John
```

2. Bracket Notation:

Bracket notation is useful when the property name is dynamic or contains special characters.

 Copy code

```
console.log(person["name"]); // Output: John
```

Adding, Modifying, or Deleting Properties

- **Add or Modify:** You can add new properties or modify existing ones using dot or bracket notation.

 Copy code

```
person.job = "Developer"; // Adding new property
person.age = 31; // Modifying existing property
```

- **Delete:** Use the `delete` keyword to remove properties.

 Copy code

```
delete person.age;
console.log(person.age); // Output: undefined
```

Methods in Objects

A method is a function that is a property of an object.

 Copy code

```
const car = {
  brand: "Toyota",
  model: "Camry",
  start: function() {
    console.log("Car started");
  }
};

car.start(); // Output: Car started
```

Looping Over Object Properties

You can loop through the properties of an object using the `for...in` loop.

 Copy code

```
const person = { name: "John", age: 30, job: "Developer" };

for (const key in person) {
  if (person.hasOwnProperty(key)) {
    console.log(`${key}: ${person[key]}`);
  }
}
```

```
}  
}
```

Object Methods

1. `Object.keys()`: Returns an array of an object's own property names.

 Copy code

```
const keys = Object.keys(person);  
console.log(keys); // Output: ['name', 'age', 'job']
```

2. `Object.values()`: Returns an array of an object's own property values.

 Copy code

```
const values = Object.values(person);  
console.log(values); // Output: ['John', 30, 'Developer']
```

3. `Object.entries()`: Returns an array of key-value pairs.

 Copy code

```
const entries = Object.entries(person);  
console.log(entries);  
// Output: [['name', 'John'], ['age', 30], ['job', 'Developer']]
```

4. `Object.assign()`: Copies properties from one or more source objects to a target object.

 Copy code

```
const obj1 = { a: 1, b: 2 };  
const obj2 = { b: 3, c: 4 };  
const result = Object.assign({}, obj1, obj2);  
console.log(result); // Output: { a: 1, b: 3, c: 4 }
```

5. `Object.freeze()`: Freezes an object so its properties can't be modified.

 Copy code

```
const obj = { a: 1 };  
Object.freeze(obj);  
obj.a = 2; // This won't change anything  
console.log(obj.a); // Output: 1
```

Prototypes and Inheritance

Objects in JavaScript can inherit properties from another object through **prototypes**. Every object in JavaScript has a prototype (another object from which it inherits properties).

 Copy code

```
const animal = {
  eat() {
    console.log("Eating...");
  }
};

const dog = Object.create(animal);
dog.bark = function() {
  console.log("Barking...");
};

dog.eat(); // Output: Eating...
dog.bark(); // Output: Barking...
```

In this example, the `dog` object inherits the `eat` method from the `animal` object via prototype inheritance.

Object Constructors

You can also create objects using **constructor functions**.

 Copy code

```
function Person(name, age) {
  this.name = name;
  this.age = age;
  this.greet = function() {
    console.log("Hello, " + this.name);
  };
}

const john = new Person("John", 30);
console.log(john.name); // Output: John
john.greet(); // Output: Hello, John
```

This pattern allows you to create multiple objects with similar structure.

Summary

- **Objects** are collections of key-value pairs.
- Use **dot notation** or **bracket notation** to access or modify properties.
- Methods are functions within objects.
- Loop through objects using `for...in` or use helper methods like `Object.keys()` and `Object.entries()`.
- JavaScript objects use **prototypes** for inheritance.
- You can use **constructor functions** to create object instances.

Let me know if you'd like to dive deeper into any specific concept!

7) JavaScript Functions

In JavaScript, **functions** are fundamental building blocks that allow you to encapsulate reusable pieces of code. A function is a block of code designed to perform a particular task. Functions can be defined, invoked (called), and passed around like any other value.

Types of Functions in JavaScript

1. Function Declaration

A function declaration defines a function with the `function` keyword, followed by a name and optional parameters. Function declarations are **hoisted**, meaning they can be called before they are defined in the code.

 Copy code

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
console.log(greet("Alice")); // Output: Hello, Alice!
```

2. Function Expression

A function expression is a function that is assigned to a variable. Unlike function declarations, function expressions are **not hoisted**, so they cannot be called before they are defined.

 Copy code

```
const greet = function(name) {  
  return `Hello, ${name}!`;  
};  
  
console.log(greet("Bob")); // Output: Hello, Bob!
```

3. Arrow Function (ES6)

Arrow functions provide a more concise syntax and have some differences in how they handle the `this` keyword (more on this below). Arrow functions are commonly used in modern JavaScript.

 Copy code

```
const greet = (name) => `Hello, ${name}!`;  
  
console.log(greet("Charlie")); // Output: Hello, Charlie!
```

Basic Syntax of Functions

1. Function Declaration Syntax

 Copy code

```
function functionName(parameters) {  
  // code to be executed  
  return someValue; // optional return statement  
}
```

2. Arrow Function Syntax

 Copy code

```
const functionName = (parameters) => {  
  // code to be executed  
  return someValue; // optional return statement  
};  
  
// Shorter version for single expressions  
const add = (a, b) => a + b;
```

Parameters and Arguments

1. **Parameters:** Parameters are the placeholders listed in the function definition.

2. **Arguments:** Arguments are the actual values passed to the function when calling it.

 Copy code

```
function add(a, b) {  
  return a + b;  
}  
  
console.log(add(3, 4)); // Output: 7
```

Default Parameters

You can assign default values to function parameters, which will be used if no argument is provided during the function call.

 Copy code

```
function greet(name = "Guest") {  
  return `Hello, ${name}!`;  
}  
  
console.log(greet()); // Output: Hello, Guest!  
console.log(greet("David")); // Output: Hello, David!
```

Rest Parameters (`...`)

Rest parameters allow you to represent an indefinite number of arguments as an array. This is useful when you don't know how many arguments will be passed to the function.

 Copy code

```
function sum(...numbers) {  
  return numbers.reduce((acc, curr) => acc + curr, 0);  
}  
  
console.log(sum(1, 2, 3, 4)); // Output: 10
```

Anonymous Functions

Anonymous functions are functions without a name. They are often used in situations where a function is passed as an argument, such as in event handlers or callbacks.

 Copy code

```
setTimeout(function() {  
  console.log("This will be executed after 1 second");  
}, 1000);
```

Immediately Invoked Function Expression (IIFE)

An IIFE is a function that runs immediately after it is defined. It's commonly used to avoid polluting the global scope.

 Copy code

```
(function() {  
  console.log("I am an IIFE!");  
})();
```

Returning Functions

Functions can return other functions, allowing you to create function factories or currying.

 Copy code

```
function createMultiplier(multiplier) {  
  return function(value) {  
    return value * multiplier;  
  };  
}  
  
const double = createMultiplier(2);  
console.log(double(5)); // Output: 10
```

Closures

A closure is a function that retains access to its lexical environment even when the function is executed outside of its original scope. Closures allow a function to access variables from an outer function after the outer function has returned.

 Copy code

```
function outer() {  
  let count = 0;  
  return function() {  
    count++;  
    return count;  
  };  
}
```

```
};  
}  
  
const counter = outer();  
console.log(counter()); // Output: 1  
console.log(counter()); // Output: 2
```

Higher-Order Functions

A higher-order function is a function that takes another function as an argument, or returns a function. This concept is central to functional programming.

 Copy code

```
function processArray(arr, callback) {  
  const result = [];  
  for (let i = 0; i < arr.length; i++) {  
    result.push(callback(arr[i]));  
  }  
  return result;  
}  
  
const doubled = processArray([1, 2, 3], (x) => x * 2);  
console.log(doubled); // Output: [2, 4, 6]
```

`this` Keyword in Functions

The behavior of the ``this`` keyword is different in regular functions and arrow functions.

Regular Functions:

In regular functions, ``this`` refers to the object that called the function, or the global object in non-strict mode.

 Copy code

```
const person = {  
  name: "Alice",  
  greet: function() {  
    console.log(this.name); // 'this' refers to the person object  
  }  
};  
  
person.greet(); // Output: Alice
```

Arrow Functions:

In arrow functions, `this` is **lexically scoped**, meaning it refers to the `this` value of the surrounding (parent) scope at the time of definition.

Copy code

```
const person = {
  name: "Bob",
  greet: () => {
    console.log(this.name); // 'this' refers to the surrounding scope
  }
};

person.greet(); // Output: undefined (because `this` is lexically bound)
```

Function Scope

- **Local Scope:** Variables declared inside a function are accessible only within that function.
- **Global Scope:** Variables declared outside any function are accessible globally.

Copy code

```
let globalVar = "Global";

function testScope() {
  let localVar = "Local";
  console.log(globalVar); // Accessible
  console.log(localVar); // Accessible
}

testScope();
console.log(globalVar); // Accessible
console.log(localVar); // Error: localVar is not defined
```

Asynchronous Functions

JavaScript functions can handle asynchronous operations using **callbacks**, **promises**, and **async/await**.

Callback Example:

Copy code

```
function fetchData(callback) {
  setTimeout(() => {
```

```

        callback("Data fetched!");
    }, 1000);
}

fetchData((data) => {
    console.log(data); // Output: Data fetched! (after 1 second)
});

```

Promise Example:

Copy code

```

function fetchData() {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            resolve("Data fetched!");
        }, 1000);
    });
}

fetchData().then((data) => console.log(data)); // Output: Data fetched!

```

Async/Await Example:

Copy code

```

async function fetchData() {
    return "Data fetched!";
}

async function displayData() {
    const data = await fetchData();
    console.log(data); // Output: Data fetched!
}

displayData();

```

Summary

- Functions are essential for reusing code and structuring JavaScript applications.
- **Function Declarations** are hoisted, while **Function Expressions** are not.
- **Arrow Functions** offer a concise syntax and different behavior with `this``.
- JavaScript supports **closures**, **higher-order functions**, and **asynchronous functions**.

- Functions can be used as values, passed as arguments, or returned from other functions.

Would you like to explore any specific aspects further?

8) merge objects in java script

In JavaScript, you can merge objects using several methods, depending on your specific needs. Below are some common ways to merge objects:

1. Using `Object.assign()`

`Object.assign()` copies all enumerable properties from one or more source objects to a target object.

 Copy code

```
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };

const mergedObj = Object.assign({}, obj1, obj2);

console.log(mergedObj); // Output: { a: 1, b: 3, c: 4 }
```

Note: If there are properties with the same key, the later object's properties will overwrite those from the earlier objects.

2. Using Spread Operator (`...`)

The spread operator (`...`) is a concise way to merge objects.

 Copy code

```
const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };

const mergedObj = { ...obj1, ...obj2 };

console.log(mergedObj); // Output: { a: 1, b: 3, c: 4 }
```

This method provides the same result as `Object.assign()` but is more concise.

3. Deep Merge with Recursion

The above methods only perform a **shallow merge**, meaning they don't merge nested objects. For deep merging, you can write a recursive function.

Copy code

```
function deepMerge(target, source) {
  for (let key in source) {
    if (source[key] instanceof Object && key in target) {
      Object.assign(source[key], deepMerge(target[key], source[key]));
    }
  }
  // Merge both objects into target
  return { ...target, ...source };
}

const obj1 = { a: 1, b: { x: 10, y: 20 } };
const obj2 = { b: { y: 30, z: 40 }, c: 5 };

const mergedObj = deepMerge(obj1, obj2);

console.log(mergedObj);
// Output: { a: 1, b: { x: 10, y: 30, z: 40 }, c: 5 }
```

This function merges objects deeply, ensuring that nested objects are combined rather than overwritten.

4. Using Lodash's `_.merge` (External Library)

If you're using a utility library like **Lodash**, it has a built-in method called `_.merge()` that performs deep merging.

Copy code

```
// First, install lodash: npm install lodash

const _ = require('lodash');

const obj1 = { a: 1, b: { x: 10, y: 20 } };
const obj2 = { b: { y: 30, z: 40 }, c: 5 };

const mergedObj = _.merge({}, obj1, obj2);

console.log(mergedObj);
// Output: { a: 1, b: { x: 10, y: 30, z: 40 }, c: 5 }
```

Lodash's `_.merge()` is ideal for deep merging, and it handles many edge cases you might not want to write manually.

Summary

- Use `Object.assign()` or the spread operator (`...`) for **shallow merges**.
- For **deep merges**, write a recursive function or use libraries like **Lodash**.

9) Difference between function and arrow functions

Function expressions and arrow functions in JavaScript have some key differences in terms of syntax, behavior, and how they handle certain aspects of the function context. Here's a detailed comparison:

1. Syntax

- **Function Expression:**

 Copy code

```
const sum = function(a, b) {  
  return a + b;  
};
```

- **Arrow Function:**

 Copy code

```
const sum = (a, b) => a + b;
```

Arrow functions provide a more concise syntax, especially for functions with a single expression.

2. `this` Context

- **Function Expression:**
- Regular functions have their own `this` context. The value of `this` is determined by how the function is called (i.e., the object that invokes the function).
- `this` can be dynamically bound using `.call()`, `.apply()`, or `.bind()`.

 Copy code

```
function RegularFunction() {
  this.value = 42;
  setTimeout(function() {
    console.log(this.value); // `this` refers to the global object or
  }, 1000);
}
new RegularFunction(); // Logs: undefined
```

- **Arrow Function:**
- Arrow functions do not have their own `this` context. Instead, they capture `this` from the surrounding lexical scope at the time they are created.
- This is useful for preserving the context in callbacks and event handlers.

Copy code

```
function ArrowFunction() {
  this.value = 42;
  setTimeout(() => {
    console.log(this.value); // `this` refers to the instance of ArrowFunction
  }, 1000);
}
new ArrowFunction(); // Logs: 42
```

3. `arguments` Object

- **Function Expression:**
- Regular functions have their own `arguments` object that contains the list of arguments passed to the function.

Copy code

```
function regularFunction() {
  console.log(arguments); // Logs the arguments passed to the function
}
regularFunction(1, 2, 3); // Logs: [1, 2, 3]
```

- **Arrow Function:**
- Arrow functions do not have their own `arguments` object. If you need access to arguments, you must use a regular function or use the rest parameter syntax

(`...args`).

Copy code

```
const arrowFunction = () => {  
  console.log(arguments); // ReferenceError: arguments is not defined  
};  
arrowFunction(1, 2, 3);
```

4. Constructors

- Function Expression:
- Regular functions can be used as constructors with the `new` keyword to create instances of objects.

Copy code

```
function Person(name) {  
  this.name = name;  
}  
const person = new Person('Alice');  
console.log(person.name); // Logs: Alice
```

- Arrow Function:
- Arrow functions cannot be used as constructors and will throw an error if used with `new`.

Copy code

```
const Person = (name) => {  
  this.name = name; // `this` is not bound correctly  
};  
const person = new Person('Alice'); // TypeError: Person is not a constructor
```

5. `super` Keyword

- Function Expression:
- Regular functions can use `super` to call methods from a parent class if used in a class context.

Copy code

```

class Parent {
  greet() {
    return 'Hello';
  }
}

class Child extends Parent {
  greet() {
    return super.greet() + ' World';
  }
}

const child = new Child();
console.log(child.greet()); // Logs: Hello World

```

- Arrow Function:
- Arrow functions do not have their own `super` binding. They cannot be used with `super` in a class context.

Copy code

```

class Parent {
  greet() {
    return 'Hello';
  }
}

class Child extends Parent {
  greet = () => super.greet() + ' World'; // SyntaxError: Unexpected su
}

const child = new Child();
console.log(child.greet()); // This will not work due to syntax error

```

6. Method Definitions

- Function Expression:
- Functions defined as methods in objects using regular function syntax.

Copy code

```

const obj = {
  value: 42,
  getValue: function() {

```

```
        return this.value;
    }
};
console.log(obj.getValue()); // Logs: 42
```

- **Arrow Function:**
- Arrow functions cannot be used as methods in objects because they do not bind their own `this`.

 Copy code

```
const obj = {
  value: 42,
  getValue: () => this.value // `this` does not refer to `obj`
};
console.log(obj.getValue()); // Logs: undefined
```

Summary:

- **Arrow Functions** provide a concise syntax and capture `this` from their lexical scope, making them useful for certain contexts, particularly for callbacks.
- **Function Expressions** have their own `this` and `arguments`, can be used as constructors, and can handle the `super` keyword in class methods.

Choose between them based on whether you need to manage `this` context, use `arguments`, or create instances.

10) explain let obj={ id:1, name:"xyz", getName:() => { return this.name }, getMyFunction(){ return this.name } } console.log(obj.getName()) console.log(obj.getMyFunction())

Let's break down the code and understand the behavior of the arrow function `getName` and the regular method `getMyFunction` in the object `obj`:

Code:

 Copy code

```
let obj = {
  id: 1,
  name: "xyz",
  getName: () => {
    return this.name;
  },
  getMyFunction: function() {
    return this.name;
  }
};
```

```
    getMyFunction() {  
        return this.name;  
    }  
}  
  
console.log(obj.getName()); // Output?  
console.log(obj.getMyFunction()); // Output?
```

Key Points:

1. Arrow Functions (`getName``):

- Arrow functions **do not** have their own `this`` context.
- `this`` in arrow functions refers to the value of `this`` in the **lexical scope** (the scope in which the arrow function was defined).
- In this case, the arrow function `getName`` is defined within the `obj`` object, but its `this`` will not refer to the `obj`` itself. Instead, `this`` refers to the global object (`window`` in browsers or `global`` in Node.js) because the lexical `this`` at the time of the arrow function's creation was the global scope.

2. Regular Method (`getMyFunction``):

- Regular methods have their own `this`` context, and `this`` inside a regular method refers to the object that called the method.
- In this case, `getMyFunction`` is a regular method, and `this`` will refer to `obj`` because `obj`` is the calling object.

Behavior Breakdown:

`obj.getName()``

The `getName`` method is defined as an arrow function. Since arrow functions do not have their own `this`` context, the `this`` inside `getName`` refers to the global object (`window`` in browsers). The global object doesn't have a `name`` property, so `this.name`` is `undefined``.

`obj.getMyFunction()``

The `getMyFunction`` method is a regular function, so its `this`` refers to `obj``, the object that is calling the method. The `name`` property exists in `obj`` and has the value `"xyz"``. Therefore, `getMyFunction`` returns `"xyz"``.

Output:

Copy code

```
console.log(obj.getName()); // undefined
console.log(obj.getMyFunction()); // "xyz"
```

Explanation of Outputs:

- `obj.getName()` logs `undefined` because `this` in the arrow function points to the global object, where `this.name` is undefined.
- `obj.getMyFunction()` logs `"xyz"` because `this` in the regular method refers to the `obj` object, where `this.name` is `"xyz"`.

How to Fix `getName` to Work as Expected:

If you want `getName` to behave like a regular method and access the `name` property of `obj`, you should not use an arrow function. You can define it as a regular method like this:

Copy code

```
let obj = {
  id: 1,
  name: "xyz",
  getName() {
    return this.name;
  },
  getMyFunction() {
    return this.name;
  }
}

console.log(obj.getName()); // "xyz"
console.log(obj.getMyFunction()); // "xyz"
```

Now both methods will return `"xyz"` because `this` will correctly refer to `obj`.

11) why arrow functions

Arrow functions in JavaScript offer a more concise syntax compared to regular functions and have different behavior in terms of how they handle the `this` context. Here's an explanation of why you might use arrow functions:

Key Features of Arrow Functions:

1. **Concise Syntax:** Arrow functions allow for a shorter syntax, especially when you don't need to define function bodies with multiple statements.

Copy code

```
// Traditional function
const add = function(a, b) {
  return a + b;
};

// Arrow function equivalent
const add = (a, b) => a + b;
```

2. **`this` Binding:** Arrow functions do **not** have their own `this` context. Instead, they inherit `this` from the surrounding lexical scope. In contrast, regular functions bind `this` to the object calling the function, or in some cases (such as event handlers or `setTimeout`), `this` may be bound to the global object (`window` in browsers).

This makes arrow functions useful in situations where you want to preserve the value of `this`, especially when working with methods or event handlers.

Example:

Copy code

```
function Timer() {
  this.seconds = 0;
  setInterval(function() {
    this.seconds++; // 'this' refers to the global object (or undefined)
    console.log(this.seconds); // Logs NaN or undefined
  }, 1000);
}
new Timer();

// Fix with arrow function
function Timer() {
  this.seconds = 0;
  setInterval(() => {
    this.seconds++; // 'this' correctly refers to the instance of Timer
    console.log(this.seconds); // Logs 1, 2, 3, etc.
  }, 1000);
}
new Timer();
```

3. **Lexical `this`**: Since arrow functions don't create their own `this`, the value of `this` inside an arrow function remains the same as in the context where the arrow function is defined. This eliminates the need to use `var self = this` or `.bind(this)` to maintain `this` context in nested functions.
4. **No `arguments` Object**: Arrow functions do not have their own `arguments` object (a built-in object in regular functions that contains the arguments passed to the function). If you need the `arguments` object, you must use a regular function, or you can use the rest parameter (`...args`) to explicitly capture arguments.

```
Copy code

function regularFunc() {
  console.log(arguments); // Works in regular functions
}

const arrowFunc = () => {
  console.log(arguments); // ReferenceError: arguments is not defined
};
```

5. **Usage in Callbacks and Closures**: Arrow functions are commonly used in callbacks (like in `setTimeout` or array methods) or when you need closures because they are often simpler to write and don't change `this`.

In Your Example (`setTimeout`):

You used an arrow function inside `setTimeout`:

```
Copy code

setTimeout(() => {
  console.log("i=" + i);
}, 500);
```

Here, the arrow function helps maintain the scope and doesn't introduce a new `this` context. However, in this particular example, the use of `this` is not a concern, so the main advantage here is the concise syntax. Arrow functions are often preferred in callbacks like this because they are shorter and easier to read.

**12) `for(var i=0;i<5;i++){ setTimeout(() => { console.log("i="+i) }, 500) }`
explain**

In this code, you're using a `for` loop to schedule five `setTimeout` calls, each of which will log the value of `i` after 500 milliseconds. However, the value of `i` will always be `5` in the output for all iterations. Here's why:

1. The `for` loop runs synchronously, so by the time the 500 milliseconds have passed and the callbacks inside `setTimeout` are executed, the `for` loop will have completed its iterations, and the value of `i` will be `5`.
2. The `setTimeout` function doesn't capture the value of `i` at the time it's called. Instead, it references the variable `i` in its outer scope. Since JavaScript variables declared with `var` are function-scoped (not block-scoped), the same `i` is shared across all iterations. When the timeouts finally run, the loop has already finished, so `i` has the value `5`.

Output:

 Copy code

```
i=5  
i=5  
i=5  
i=5  
i=5
```

Fix:

To capture the value of `i` in each iteration, you can use `let` instead of `var`. The `let` keyword has block scope, meaning a new `i` is created for each iteration.

 Copy code

```
for (let i = 0; i < 5; i++) {  
  setTimeout(() => {  
    console.log("i=" + i);  
  }, 500);  
}
```

This will correctly log the values of `i` from 0 to 4.

Output with `let`:

 Copy code

```
i=0  
i=1  
i=2  
i=3  
i=4
```

13) const obj1 = { a: 1, b: { c: 2, d: [3, 4] } }; const obj2 = { b: { c: 5, d: [6] }, e: 7 }; const merged = deepMerge(obj1, obj2); console.log(merged);

To achieve a deep merge of two objects like `obj1` and `obj2`, we need to recursively merge nested objects and arrays. Here's a possible implementation of `deepMerge`:

 Copy code

```
function deepMerge(target, source) {  
  for (let key in source) {  
    if (source.hasOwnProperty(key)) {  
      if (typeof source[key] === 'object' && source[key] !== null && !Array.isArray(source[key]) && (!target[key] || typeof target[key] !== 'object')) {  
        target[key] = {};  
      }  
      deepMerge(target[key], source[key]);  
    } else if (Array.isArray(source[key])) {  
      // For arrays, we can choose to either replace or concatenate the arrays.  
      // Here, I will concatenate arrays from the source and target.  
      if (!Array.isArray(target[key])) {  
        target[key] = [];  
      }  
      target[key] = target[key].concat(source[key]);  
    } else {  
      target[key] = source[key];  
    }  
  }  
  return target;  
}  
  
const obj1 = {  
  a: 1,  
  b: {  
    c: 2,  
    d: [3, 4]  
  }  
};  
const obj2 = {  
  b: {
```

```
    c: 5,  
    d: [6]  
  },  
  e: 7  
};  
  
const merged = deepMerge(obj1, obj2);  
console.log(merged);
```

Output:

 Copy code

```
{  
  "a": 1,  
  "b": {  
    "c": 5,  
    "d": [3, 4, 6]  
  },  
  "e": 7  
}
```

Explanation:

- Primitive values like numbers are overwritten.
- Arrays are concatenated (e.g., ``d: [3, 4]`` and ``d: [6]`` becomes ``d: [3, 4, 6]``).
- Nested objects are merged recursively.